

CS 4644/7643: Deep Learning

Fall 2026

Problem Set 2

Instructor: Danfei Xu

TAs: Rohan Bansal, Uzu (Chris) Lee, Alfred Cueva
Kun-Lin Hsieh, Chuye Zhang, Liqian Ma, Robert Azarcon
Dhruv Ahuja, Xinchun Yin

Discussions: <https://piazza.com/class/ms43b2tjxa63lq#>

Deadline: Wednesday, October 7 2026, 11:59 pm

Student Name: Venkata Aashutosh Aripirala **GT Email:** av84@gatech.edu **GT ID:** 904173725

Instructions

1. We will be using Gradescope to collect your assignments. Please read the following instructions for submitting to Gradescope carefully!
 - For the **HW2 Theory** component on Gradescope, upload one single PDF containing the answers to all the theory questions. **However, the solution to each problem/subproblem must be on a separate page. When submitting to Gradescope, please make sure to mark the page(s) corresponding to each problem/subproblem.**
 - For the **HW2 Coding - Part 1** on Gradescope, please use the `collect_submission.py` script provided and upload **hw2_part1_code_submission.zip**
 - For the **HW2 Coding - Part 2** on Gradescope, please use the `collect_submission.py` script provided and upload **hw2_part2_code_submission.zip**
 - For the **HW2 Notebook** component on Gradescope, please use the `collect_submission.py` script provided and upload the resulting **hw2_notebook_submission.pdf** on Gradescope.
 - Note: This is a large class and Gradescope's assignment segmentation features are essential. Failure to follow these instructions may result in parts of your assignment not being graded. We will not entertain regrading requests for failure to follow instructions.
2. $\text{\LaTeX}$ 'd solutions are strongly encouraged, but scanned handwritten copies are acceptable. Hard copies are **not** accepted.
3. We generally encourage you to collaborate with other students.

You may talk to a friend, discuss the questions and potential directions for solving them. However, you need to write your own solutions and code separately, and *not* as a group activity. Please list the students you collaborated with.

1 Collaborators [0.5 points]

Please list your collaborators and assign this list to the corresponding section of the outline on Gradescope. If you don't have any collaborators, please write 'None' and assign it to the corresponding section of the Gradescope submission regardless. Not assigning this question on Gradescope or leaving it blank will be marked as zero.

Solution: None

2 Gradient Descent

1. [3 points]

We often use iterative optimization algorithms such as Gradient Descent to find $\mathbf{w}$ that minimizes a loss function $f(\mathbf{w})$. Recall that in gradient descent, we start with an initial value of $\mathbf{w}$ (say $\mathbf{w}^{(1)}$) and iteratively take a step in the direction of the negative of the gradient of the objective function *i.e.*

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla f(\mathbf{w}^{(t)}) \quad (1)$$

for learning rate $\eta > 0$.

In this question, we will develop a slightly deeper understanding of this update rule, in particular for minimizing a convex function $f(\mathbf{w})$. Note: this analysis will not directly carry over to training neural networks since loss functions for training neural networks are typically not convex, but this will (a) develop intuition and (b) provide a starting point for research in non-convex optimization (which is beyond the scope of this class).

Recall the first-order Taylor approximation of f at $\mathbf{w}^{(t)}$:

$$f(\mathbf{w}) \approx f(\mathbf{w}^{(t)}) + \langle \mathbf{w} - \mathbf{w}^{(t)}, \nabla f(\mathbf{w}^{(t)}) \rangle \quad (2)$$

When f is convex, this approximation forms a lower bound of f , *i.e.*

$$f(\mathbf{w}) \geq \underbrace{f(\mathbf{w}^{(t)}) + \langle \mathbf{w} - \mathbf{w}^{(t)}, \nabla f(\mathbf{w}^{(t)}) \rangle}_{\text{affine lower bound to } f(\cdot)} \quad \forall \mathbf{w} \quad (3)$$

Since this approximation is a ‘simpler’ function than $f(\cdot)$, we could consider minimizing the approximation instead of $f(\cdot)$. Two immediate problems: (1) the approximation is affine (thus unbounded from below) and (2) the approximation is faithful for $\mathbf{w}$ close to $\mathbf{w}^{(t)}$. To solve both problems, we add a squared ℓ_2 *proximity term* to the approximation minimization:

$$\underset{\mathbf{w}}{\operatorname{argmin}} \underbrace{f(\mathbf{w}^{(t)}) + \langle \mathbf{w} - \mathbf{w}^{(t)}, \nabla f(\mathbf{w}^{(t)}) \rangle}_{\text{affine lower bound to } f(\cdot)} + \underbrace{\frac{\lambda}{2}}_{\text{trade-off}} \underbrace{\|\mathbf{w} - \mathbf{w}^{(t)}\|^2}_{\text{proximity term}} \quad (4)$$

Notice that the optimization problem above is an unconstrained quadratic programming problem, meaning that it can be solved in closed form (hint: gradients).

What is the solution $\mathbf{w}^*$ of the above optimization? What does that tell you about the gradient descent update rule? What is the relationship between λ and η ?

Solution: 1. Derivation of the closed-form solution $\mathbf{w}^*$:

Let the objective function to minimize be denoted by $g(\mathbf{w})$:

$$g(\mathbf{w}) = f(\mathbf{w}^{(t)}) + \langle \mathbf{w} - \mathbf{w}^{(t)}, \nabla f(\mathbf{w}^{(t)}) \rangle + \frac{\lambda}{2} \|\mathbf{w} - \mathbf{w}^{(t)}\|^2$$

Since $g(\mathbf{w})$ is a strictly convex quadratic function in $\mathbf{w}$ (with $\lambda > 0$), its global minimum occurs where the gradient with respect to $\mathbf{w}$ is equal to $\mathbf{0}$.

Taking the gradient with respect to $\mathbf{w}$:

$$\nabla_{\mathbf{w}} g(\mathbf{w}) = \nabla f(\mathbf{w}^{(t)}) + \mathbf{0} + \lambda(\mathbf{w} - \mathbf{w}^{(t)})$$

Setting the gradient to $\mathbf{0}$ since $\mathbf{w}^*$ is the minima of the curve of the optimization function :

$$\begin{aligned}\nabla f(\mathbf{w}^{(t)}) + \lambda(\mathbf{w}^* - \mathbf{w}^{(t)}) &= \mathbf{0} \\ \lambda(\mathbf{w}^* - \mathbf{w}^{(t)}) &= -\nabla f(\mathbf{w}^{(t)}) \\ \mathbf{w}^* &= \mathbf{w}^{(t)} - \frac{1}{\lambda} \nabla f(\mathbf{w}^{(t)})\end{aligned}$$

2. Interpretation of the Gradient Descent update rule:

Comparing the closed-form solution $\mathbf{w}^*$ to the gradient descent update rule:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla f(\mathbf{w}^{(t)})$$

This reveals that gradient descent can be viewed as a proximal algorithm: at each step t , the update optimizes a local first-order Taylor approximation of the function while simultaneously penalizing deviation from the current estimate $\mathbf{w}^{(t)}$ via an ℓ_2 proximity term. This prevents the update from taking arbitrarily large steps into regions where the linear approximation no longer holds.

3. Relationship between λ and η :

Equating $\mathbf{w}^*$ directly to $\mathbf{w}^{(t+1)}$ yields:

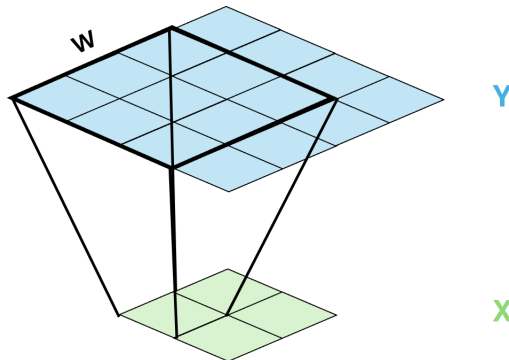
$$\eta = \frac{1}{\lambda} \quad \text{or equivalently} \quad \lambda = \frac{1}{\eta}$$

The learning rate η is inversely proportional to the regularization weight λ . A larger proximity penalty λ restricts the step size, producing a smaller, more conservative learning rate η . Conversely, a smaller proximity penalty λ allows larger updates, corresponding to a larger learning rate η .

3 Convolutions

2. [4 points]

Transposed Convolution is an upsampling operation used in tasks such as image segmentation and generative models. Unlike standard convolution which downsamples the input, transposed convolution expands the spatial dimensions. The following figure shows transposed convolution with input $x \in \mathbb{R}^{2 \times 2}$, kernel $w \in \mathbb{R}^{3 \times 3}$, and output $y \in \mathbb{R}^{4 \times 4}$. You may find this [link](#) helpful for understanding the operation in-depth.



Problem: Assume input $x \in \mathbb{R}^{3 \times 3}$ and kernel $w \in \mathbb{R}^{3 \times 3}$. When transposed convolution with stride 2 is applied, the resulting output y will have dimensions 7×7 . The gradient of the loss L with respect to the output y is given as $\delta = \frac{\partial L}{\partial y} \in \mathbb{R}^{7 \times 7}$.

(a) [1 point] Compute output y with following given input x and kernel w .

Given:

$$w = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 2 \\ 3 & 4 & 5 \end{bmatrix}, \quad x = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Compute:

$$y = \begin{bmatrix} & & & & & & \\ & & & & & & \\ & & & & & & \\ & & & & & & \\ & & & & & & \\ & & & & & & \\ & & & & & & \end{bmatrix}$$

Solution: In a 2D transposed convolution with stride $s = 2$ and kernel size 3×3 , each input scalar $x[i, j]$ (for $i, j \in \{0, 1, 2\}$) scales the entire 3×3 kernel w and scatters it onto the output grid $y \in \mathbb{R}^{7 \times 7}$ starting at top-left index $(2i, 2j)$. Mathematically, for any output entry $y[r, c]$ with $0 \leq r, c \leq 6$:

$$y[r, c] = \sum_{i=0}^2 \sum_{j=0}^2 x[i, j] \cdot w[r - 2i, c - 2j] \cdot \mathbb{I}(0 \leq r - 2i \leq 2) \cdot \mathbb{I}(0 \leq c - 2j \leq 2)$$

Since all entries of x are 1 ($x[i, j] = 1$), each output position is simply the sum of overlapping kernel elements $w[p, q]$ where $p = r - 2i$ and $q = c - 2j$.

Because the 2D operation factors independently along rows and columns, we determine the active kernel rows p contributing to each output row r :

- $r = 0$: Only $i = 0$ contributes $\implies p = 0 - 0 = 0$ (Row 0 of w).
- $r = 1$: Only $i = 0$ contributes $\implies p = 1 - 0 = 1$ (Row 1 of w).
- $r = 2$: $i = 0$ gives $p = 2$; $i = 1$ gives $p = 0 \implies$ (Row 2 of w) + (Row 0 of w).
- $r = 3$: Only $i = 1$ contributes $\implies p = 3 - 2 = 1$ (Row 1 of w).
- $r = 4$: $i = 1$ gives $p = 2$; $i = 2$ gives $p = 0 \implies$ (Row 2 of w) + (Row 0 of w).
- $r = 5$: Only $i = 2$ contributes $\implies p = 5 - 4 = 1$ (Row 1 of w).
- $r = 6$: Only $i = 2$ contributes $\implies p = 6 - 4 = 2$ (Row 2 of w).

Given the kernel rows: $\text{row}_0(w) = [0 \ 0 \ 0]$, $\text{row}_1(w) = [0 \ 1 \ 2]$, $\text{row}_2(w) = [3 \ 4 \ 5]$.

Applying the identical column mapping along the horizontal axis:

- For rows receiving $\text{row}_0(w)$ (Row 0): All entries are 0 $\implies [0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0]$.

- For rows receiving $\text{row}_1(w)$ (Rows 1, 3, 5):

$$\begin{aligned}
c = 0 : w[1, 0] &= 0 \\
c = 1 : w[1, 1] &= 1 \\
c = 2 : w[1, 2] + w[1, 0] &= 2 + 0 = 2 \\
c = 3 : w[1, 1] &= 1 \\
c = 4 : w[1, 2] + w[1, 0] &= 2 + 0 = 2 \\
c = 5 : w[1, 1] &= 1 \\
c = 6 : w[1, 2] &= 2 \\
\implies [0 & 1 \ 2 \ 1 \ 2 \ 1 \ 2]
\end{aligned}$$

- For rows receiving $\text{row}_2(w) + \text{row}_0(w)$ (Rows 2, 4, 6): Since $\text{row}_0(w)$ is all zeros, this is solely determined by $\text{row}_2(w)$:

$$\begin{aligned}
c = 0 : w[2, 0] &= 3 \\
c = 1 : w[2, 1] &= 4 \\
c = 2 : w[2, 2] + w[2, 0] &= 5 + 0 = 5 \\
c = 3 : w[2, 1] &= 4 \\
c = 4 : w[2, 2] + w[2, 0] &= 5 + 0 = 5 \\
c = 5 : w[2, 1] &= 4 \\
c = 6 : w[2, 2] &= 5 \\
\implies [3 & 4 \ 5 \ 4 \ 5 \ 4 \ 5]
\end{aligned}$$

Assembling all 7 rows gives:

$$y = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 1 & 2 & 1 & 2 \\ 3 & 4 & 5 & 4 & 5 & 4 & 5 \\ 0 & 1 & 2 & 1 & 2 & 1 & 2 \\ 3 & 4 & 5 & 4 & 5 & 4 & 5 \\ 0 & 1 & 2 & 1 & 2 & 1 & 2 \\ 3 & 4 & 5 & 4 & 5 & 4 & 5 \end{bmatrix}$$

(b) **[1 point]** Assume the following simple values for the kernel w and the gradient δ . Calculate $\frac{\partial L}{\partial x}$. (Hint : It can be simplified by convolution or transposed convolution between the two given matrices)

Given:

$$w = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 2 \\ 3 & 4 & 5 \end{bmatrix}, \quad \delta = \begin{bmatrix} 0 & 0 & 0 & 2 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 2 & 0 & 0 & 0 & 0 & 0 & 2 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 2 & 0 & 0 & 0 \end{bmatrix}$$

Calculate:

$$\frac{\partial L}{\partial x} = \begin{bmatrix} \\ \\ \\ \\ \\ \\ \end{bmatrix}$$

Solution: By the chain rule, the gradient of the loss with respect to input $x[i, j]$ is given by:

$$\frac{\partial L}{\partial x[i, j]} = \sum_{r=0}^6 \sum_{c=0}^6 \frac{\partial L}{\partial y[r, c]} \frac{\partial y[r, c]}{\partial x[i, j]} = \sum_{p=0}^2 \sum_{q=0}^2 \delta[2i + p, 2j + q] \cdot w[p, q]$$

This corresponds to computing the Frobenius inner product between the kernel w and the 3×3 sub-window of δ starting at row $2i$ and column $2j$, with stride 2:

$$\frac{\partial L}{\partial x[i, j]} = \langle \delta[2i : 2i + 3, 2j : 2j + 3], w \rangle_F$$

Recall the values of w and the non-zero coordinates of δ :

$$w = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 2 \\ 3 & 4 & 5 \end{bmatrix}$$

Computing each entry of $\frac{\partial L}{\partial x}$:

- $(i = 0, j = 0)$: Sub-window $\delta[0..2, 0..2]$:

$$\delta[0..2, 0..2] = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 0 \end{bmatrix} \implies \delta[1, 1] \cdot w[1, 1] = 3 \cdot 1 = 3$$

- $(i = 0, j = 1)$: Sub-window $\delta[0..2, 2..4]$:

$$\delta[0..2, 2..4] = \begin{bmatrix} 0 & 2 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \implies \delta[0, 3] \cdot w[0, 1] = 2 \cdot 0 = 0$$

- $(i = 0, j = 2)$: Sub-window $\delta[0..2, 4..6]$:

$$\delta[0..2, 4..6] = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 0 \end{bmatrix} \implies \delta[1, 5] \cdot w[1, 1] = 3 \cdot 1 = 3$$

- $(i = 1, j = 0)$: Sub-window $\delta[2..4, 0..2]$:

$$\delta[2..4, 0..2] = \begin{bmatrix} 0 & 0 & 0 \\ 2 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \implies \delta[3, 0] \cdot w[1, 0] = 2 \cdot 0 = 0$$

- $(i = 1, j = 1)$: Sub-window $\delta[2..4, 2..4]$: All entries within this sub-window in δ are 0 $\implies 0$.

- $(i = 1, j = 2)$: Sub-window $\delta[2..4, 4..6]$:

$$\delta[2..4, 4..6] = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 2 \\ 0 & 0 & 0 \end{bmatrix} \implies \delta[3, 6] \cdot w[1, 2] = 2 \cdot 2 = 4$$

- $(i = 2, j = 0)$: Sub-window $\delta[4..6, 0..2]$:

$$\delta[4..6, 0..2] = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 0 \end{bmatrix} \implies \delta[5, 1] \cdot w[1, 1] = 3 \cdot 1 = 3$$

- $(i = 2, j = 1)$: Sub-window $\delta[4..6, 2..4]$:

$$\delta[4..6, 2..4] = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 2 & 0 \end{bmatrix} \implies \delta[6, 3] \cdot w[2, 1] = 2 \cdot 4 = 8$$

- $(i = 2, j = 2)$: Sub-window $\delta[4..6, 4..6]$:

$$\delta[4..6, 4..6] = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 0 \end{bmatrix} \implies \delta[5, 5] \cdot w[1, 1] = 3 \cdot 1 = 3$$

Thus:

$$\frac{\partial L}{\partial x} = \begin{bmatrix} 3 & 0 & 3 \\ 0 & 0 & 4 \\ 3 & 8 & 3 \end{bmatrix}$$

(c) **[1 point]** Assume the following simple values for the input x and the gradient δ . Calculate $\frac{\partial L}{\partial w}$. **Be careful that x here is different from (a).** (Hint : Similar to (b), it can be simplified by convolution or transposed convolution but it may need some manipulation on given matrix)

Given:

$$x = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}, \delta = \begin{bmatrix} 0 & 0 & 0 & 2 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 2 & 0 & 0 & 0 & 0 & 0 & 2 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 2 & 0 & 0 & 0 \end{bmatrix}$$

Calculate:

$$\frac{\partial L}{\partial w} = \begin{bmatrix} & & & & & & \\ & & & & & & \\ & & & & & & \\ & & & & & & \\ & & & & & & \\ & & & & & & \\ & & & & & & \end{bmatrix}$$

Solution: By the chain rule, each kernel weight $w[p, q]$ influences output elements at positions $(2i + p, 2j + q)$. Therefore:

$$\frac{\partial L}{\partial w[p, q]} = \sum_{i=0}^2 \sum_{j=0}^2 \delta[2i + p, 2j + q] \cdot x[i, j]$$

Given the input matrix:

$$x = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

The non-zero entries of x are located at $(i, j) \in \{(0, 0), (0, 2), (1, 1), (2, 0), (2, 2)\}$, where each non-zero value equals 1. Hence, the formula simplifies to the sum over these 5 points:

$$\frac{\partial L}{\partial w[p, q]} = \delta[p, q] + \delta[p, q + 4] + \delta[p + 2, q + 2] + \delta[p + 4, q] + \delta[p + 4, q + 4]$$

We evaluate this sum for each kernel coordinate $(p, q) \in \{0, 1, 2\} \times \{0, 1, 2\}$:

- $(p = 0, q = 0)$: $\delta[0, 0] + \delta[0, 4] + \delta[2, 2] + \delta[4, 0] + \delta[4, 4] = 0 + 0 + 0 + 0 + 0 = 0$
- $(p = 0, q = 1)$: $\delta[0, 1] + \delta[0, 5] + \delta[2, 3] + \delta[4, 1] + \delta[4, 5] = 0 + 0 + 0 + 0 + 0 = 0$
- $(p = 0, q = 2)$: $\delta[0, 2] + \delta[0, 6] + \delta[2, 4] + \delta[4, 2] + \delta[4, 6] = 0 + 0 + 0 + 0 + 0 = 0$
- $(p = 1, q = 0)$: $\delta[1, 0] + \delta[1, 4] + \delta[3, 2] + \delta[5, 0] + \delta[5, 4] = 0 + 0 + 0 + 0 + 0 = 0$
- $(p = 1, q = 1)$: $\delta[1, 1] + \delta[1, 5] + \delta[3, 3] + \delta[5, 1] + \delta[5, 5] = 3 + 3 + 0 + 3 + 3 = 12$
- $(p = 1, q = 2)$: $\delta[1, 2] + \delta[1, 6] + \delta[3, 4] + \delta[5, 2] + \delta[5, 6] = 0 + 0 + 0 + 0 + 0 = 0$

- $(p = 2, q = 0)$: $\delta[2, 0] + \delta[2, 4] + \delta[4, 2] + \delta[6, 0] + \delta[6, 4] = 0 + 0 + 0 + 0 + 0 = 0$
- $(p = 2, q = 1)$: $\delta[2, 1] + \delta[2, 5] + \delta[4, 3] + \delta[6, 1] + \delta[6, 5] = 0 + 0 + 0 + 0 + 0 = 0$
- $(p = 2, q = 2)$: $\delta[2, 2] + \delta[2, 6] + \delta[4, 4] + \delta[6, 2] + \delta[6, 6] = 0 + 0 + 0 + 0 + 0 = 0$

Thus:

$$\frac{\partial L}{\partial w} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 12 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

(d) [1 point] (Required for 7643; Extra Credit for 4644)

In the original U-Net architecture (Ronneberger et al., 2015), **Transposed Convolution** (often referred to as up-convolution) was used to upsample feature maps. However, in modern generative models (e.g., GANs), this operation can sometimes introduce grid-like patterns known as "checkerboard artifacts."

Question:

Consider a 2D Transposed Convolution with Kernel Size $k = 8$ and Stride $s = 3$.

Explain why some pixels in the output y receive contributions from more input pixels than others. Based on your explanation, **suggest a stride($\neq 1$)** to avoid this uneven overlap (i.e., to avoid checkerboard artifacts).

Solution: 1. Why uneven overlap occurs:

In transposed convolution, each input pixel paints an output footprint of width k , spaced at intervals of stride s . For any 1D output coordinate y , the number of input pixels i contributing to it corresponds to the number of valid integers i satisfying:

$$0 \leq y - s \cdot i < k \iff \frac{y - k + 1}{s} \leq i \leq \frac{y}{s}$$

When the kernel size k is not divisible by the stride s ($k \pmod s \neq 0$), the overlap pattern varies periodically depending on $y \pmod s$.

Specifically, for $k = 8$ and $s = 3$:

$$8 \pmod 3 = 2 \neq 0$$

Because $8 = 2 \times 3 + 2$, the kernel spans two full stride intervals plus a fractional remainder of 2. Consequently:

- Positions where the remainder overlaps cover $\lceil k/s \rceil = \lceil 8/3 \rceil = 3$ input pixels.
- Other positions cover only $\lfloor k/s \rfloor = \lfloor 8/3 \rfloor = 2$ input pixels.

In 2D, the receptive field is the Cartesian product of the two dimensions. Thus, some output pixels receive contributions from $3 \times 3 = 9$ input features, while neighboring pixels receive contributions from only $2 \times 2 = 4$ input features (a ratio greater than 2 : 1). This non-uniform accumulation creates periodic, grid-like intensity variations across the image, which are also called checkerboard artifacts.

2. Suggested Stride ($\neq 1$):

To achieve uniform coverage across all output positions, the kernel size k must be an exact integer multiple of the stride s , i.e the stride s must be a factor of k :

$$k \pmod s = 0$$

For $k = 8$, the integer divisors greater than 1 are $s \in \{2, 4, 8\}$.

Therefore, setting the stride to $s = 2$ (or $s = 4$) completely eliminates uneven overlap. With $s = 2$, every output location receives contributions from exactly $k/s = 8/2 = 4$ inputs in 1D ($4 \times 4 = 16$ inputs in 2D), ensuring uniform overlap and avoiding checkerboard artifacts.

4 Initialization

3. [3 points + 2 points]

Proper weight initialization is critical for training deep networks. If activations grow or shrink as they propagate through layers, training becomes unstable. In this problem, we will derive some common initialization schemes. You may find that completing the paper review for this assignment will help with this problem.

Consider a deep network with layers $h^{(l)} = W^{(l)}h^{(l-1)}$, ignoring biases for now. Here $h^{(l)} \in \mathbb{R}^{n_{\text{out}}}$ is a column vector of activations, and the weight matrix $W^{(l)} \in \mathbb{R}^{n_{\text{out}} \times n_{\text{in}}}$ maps n_{in} inputs to n_{out} outputs.

Assume:

- Inputs $h^{(0)}$ are i.i.d. with zero mean and variance $\text{Var}(h^{(0)}) = \sigma^2$
- Weights are initialized i.i.d. with zero mean and variance $\text{Var}(W^{(l)})$
- Inputs and weights are independent
- You can directly use standard probability identities for expectations and variance

(a) [1 point]

We first analyze how variance changes through a single linear layer. Show that:

$$\text{Var}(h^{(1)}) = n_{\text{in}} \cdot \text{Var}(W^{(1)}) \cdot \sigma^2$$

Solution: Each element $i \in \{1, \dots, n_{\text{out}}\}$ of the output activation vector $h^{(1)}$ is computed as:

$$h_i^{(1)} = \sum_{j=1}^{n_{\text{in}}} W_{ij}^{(1)} h_j^{(0)}$$

First, computing the expectation of $h_i^{(1)}$. Since $W_{ij}^{(1)}$ and $h_j^{(0)}$ are independent, and both have zero mean:

$$\mathbb{E}[h_i^{(1)}] = \sum_{j=1}^{n_{\text{in}}} \mathbb{E}[W_{ij}^{(1)}] \mathbb{E}[h_j^{(0)}] = \sum_{j=1}^{n_{\text{in}}} 0 \cdot 0 = 0$$

Next, because the summands $W_{ij}^{(1)} h_j^{(0)}$ are mutually independent for different j , the variance of the sum is the sum of the variances:

$$\text{Var}(h_i^{(1)}) = \sum_{j=1}^{n_{\text{in}}} \text{Var}\left(W_{ij}^{(1)} h_j^{(0)}\right)$$

For any two independent random variables X and Y with zero mean, the variance of their product is:

$$\text{Var}(XY) = \mathbb{E}[(XY)^2] - (\mathbb{E}[XY])^2 = \mathbb{E}[X^2]\mathbb{E}[Y^2] - 0 = \text{Var}(X)\text{Var}(Y)$$

Applying this identity with $X = W_{ij}^{(1)}$ and $Y = h_j^{(0)}$:

$$\text{Var}\left(W_{ij}^{(1)} h_j^{(0)}\right) = \text{Var}(W_{ij}^{(1)})\text{Var}(h_j^{(0)}) = \text{Var}(W^{(1)}) \cdot \sigma^2$$

Summing across all n_{in} independent inputs:

$$\text{Var}(h^{(1)}) = \sum_{j=1}^{n_{\text{in}}} \text{Var}(W^{(1)}) \cdot \sigma^2 = n_{\text{in}} \cdot \text{Var}(W^{(1)}) \cdot \sigma^2$$

(b) **[1 point]**

Using your result from part (a), determine what $\text{Var}(W^{(l)})$ should be to ensure the variance is preserved, i.e., $\text{Var}(h^{(1)}) = \text{Var}(h^{(0)})$. This is known as **Xavier initialization**.

Solution: We want:

$$\text{Var}(h^{(1)}) = \text{Var}(h^{(0)}) = \sigma^2$$

Substituting the expression derived in part (a):

$$n_{\text{in}} \cdot \text{Var}(W^{(1)}) \cdot \sigma^2 = \sigma^2$$

Dividing both sides by $n_{\text{in}} \cdot \sigma^2$ (for $\sigma^2 \neq 0$):

$$\text{Var}(W^{(l)}) = \frac{1}{n_{\text{in}}}$$

Thus, under Xavier initialization for the forward pass, initializing weights i.i.d. with mean 0 and variance $\frac{1}{n_{\text{in}}}$ guarantees that the activation scale remains constant across consecutive layers.

(c) **[1 point]**

Modern networks use nonlinear activations. Before deriving the appropriate initialization for ReLU networks, we need a key identity. Let x be a random variable with a symmetric distribution with zero mean. Prove that:

$$\mathbb{E}[\text{ReLU}(x)^2] = \frac{1}{2}\mathbb{E}[x^2]$$

Solution: Let $p(x)$ denote the probability density function of x . Since the distribution of x is symmetric about zero, we have $p(-x) = p(x)$ for all $x \in \mathbb{R}$.

$$\text{ReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases} \implies \text{ReLU}(x)^2 = \begin{cases} x^2 & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$

$$\begin{aligned} \mathbb{E}[\text{ReLU}(x)^2] &= \int_{-\infty}^{\infty} \text{ReLU}(x)^2 p(x) \, dx \\ &= \int_{-\infty}^0 0 \cdot p(x) \, dx + \int_0^{\infty} x^2 p(x) \, dx \\ &= \int_0^{\infty} x^2 p(x) \, dx \end{aligned}$$

Now, consider the total expectation $\mathbb{E}[x^2]$:

$$\mathbb{E}[x^2] = \int_{-\infty}^{\infty} x^2 p(x) \, dx = \int_{-\infty}^0 x^2 p(x) \, dx + \int_0^{\infty} x^2 p(x) \, dx$$

Under the change of variables $u = -x$ (with $dx = -du$), and utilizing our known symmetry $p(-u) = p(u)$:

$$\int_{-\infty}^0 x^2 p(x) \, dx = \int_{\infty}^0 (-u)^2 p(-u) (-du) = \int_0^{\infty} u^2 p(u) \, du$$

Therefore:

$$\mathbb{E}[x^2] = \int_0^{\infty} x^2 p(x) \, dx + \int_0^{\infty} x^2 p(x) \, dx = 2 \int_0^{\infty} x^2 p(x) \, dx \implies \frac{1}{2}\mathbb{E}[x^2] = \int_0^{\infty} x^2 p(x) \, dx$$

It follows directly that:

$$\mathbb{E}[\text{ReLU}(x)^2] = \int_0^{\infty} x^2 p(x) \, dx = \frac{1}{2}\mathbb{E}[x^2]$$

(d) [2 points] (Required for 7643: Extra Credit for 4644)

Now consider a network with ReLU activations: $h^{(l)} = W^{(l)} \text{ReLU}(h^{(l-1)})$. Using your result from part (c), derive the initialization variance $\text{Var}(W^{(l)})$ needed to ensure the variance is preserved, i.e., $\text{Var}(h^{(1)}) = \text{Var}(h^{(0)})$. This is known as **Kaiming initialization**.

Solution: Let $a_j^{(0)} = \text{ReLU}(h_j^{(0)})$. The i -th entry of $h^{(1)}$ is given by:

$$h_i^{(1)} = \sum_{j=1}^{n_{\text{in}}} W_{ij}^{(1)} a_j^{(0)} = \sum_{j=1}^{n_{\text{in}}} W_{ij}^{(1)} \text{ReLU}(h_j^{(0)})$$

Evaluating the expectation of $h_i^{(1)}$. Because weights $W_{ij}^{(1)}$ are independent of the inputs $h_j^{(0)}$ and have zero mean ($\mathbb{E}[W_{ij}^{(1)}] = 0$):

$$\mathbb{E}[h_i^{(1)}] = \sum_{j=1}^{n_{\text{in}}} \mathbb{E}[W_{ij}^{(1)}] \mathbb{E}[a_j^{(0)}] = 0$$

Since the mean is zero, the variance simply equals the second moment:

$$\text{Var}(h_i^{(1)}) = \mathbb{E} \left[\left(h_i^{(1)} \right)^2 \right] = \mathbb{E} \left[\left(\sum_{j=1}^{n_{\text{in}}} W_{ij}^{(1)} a_j^{(0)} \right)^2 \right]$$

Expanding the square, the cross-terms $\mathbb{E}[W_{ij}^{(1)} W_{ik}^{(1)} a_j^{(0)} a_k^{(0)}]$ vanish for $j \neq k$ because weights are mutually independent with zero mean. Thus:

$$\text{Var}(h_i^{(1)}) = \sum_{j=1}^{n_{\text{in}}} \mathbb{E} \left[\left(W_{ij}^{(1)} \right)^2 \right] \mathbb{E} \left[\left(a_j^{(0)} \right)^2 \right]$$

Since $\mathbb{E}[W_{ij}^{(1)}] = 0$, we have $\mathbb{E}[(W_{ij}^{(1)})^2] = \text{Var}(W^{(1)})$.

Now, applying the identity from (c) to $h_j^{(0)}$ (which has zero mean, and variance σ^2 , so assuming symmetric distribution given by (c), $\mathbb{E}[(h_j^{(0)})^2] = \sigma^2$):

$$\mathbb{E} \left[\left(a_j^{(0)} \right)^2 \right] = \mathbb{E} \left[\text{ReLU}(h_j^{(0)})^2 \right] = \frac{1}{2} \mathbb{E} \left[\left(h_j^{(0)} \right)^2 \right] = \frac{1}{2} \sigma^2$$

Substituting this back into the variance summation:

$$\text{Var}(h^{(1)}) = \sum_{j=1}^{n_{\text{in}}} \text{Var}(W^{(1)}) \cdot \frac{1}{2} \sigma^2 = n_{\text{in}} \cdot \text{Var}(W^{(1)}) \cdot \frac{1}{2} \sigma^2$$

To ensure variance is preserved, we require $\text{Var}(h^{(1)}) = \text{Var}(h^{(0)}) = \sigma^2$:

$$\begin{aligned} n_{\text{in}} \cdot \text{Var}(W^{(1)}) \cdot \frac{1}{2} \sigma^2 &= \sigma^2 \\ n_{\text{in}} \cdot \text{Var}(W^{(1)}) \cdot \frac{1}{2} &= 1 \\ \text{Var}(W^{(l)}) &= \frac{2}{n_{\text{in}}} \end{aligned}$$

Thus, for networks with ReLU activations, Kaiming initialization requires:

$$\text{Var}(W^{(l)}) = \frac{2}{n_{\text{in}}}$$

The factor of 2, different from our initial proof for linear networks accounts for the fact that ReLU sets approximately half of the activations to zero, halving the expected power of the forward signal at each layer.

5 Paper Review

4. [4 points]

‘Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification’ by Kaiming He, et al.

The paper examines the fundamental importance of rectifier neural networks and proper initialization for training very deep networks. The study also presents, using experimental results, the PReLU networks (PReLU-nets), which was the first neural network to surpass human-level performance on ImageNet classification. The paper can be viewed [here](#).

The evaluation rubric for this section is as follows:

- (e) [2 points] Briefly summarize the key contributions, strengths and weaknesses of this paper.
- (f) [2 points] What is your personal takeaway from this paper? This could be expressed either in terms of relating the approaches adopted in this paper to your traditional understanding of image classification networks, or potential future directions of research in the area which the authors haven’t addressed, or anything else that struck you as being noteworthy.

Guidelines: Please restrict your reviews to no more than 350 words (total length for answers to both the above questions).

Solution: Key Contributions, Strengths, and Weaknesses: The paper makes two primary contributions. First, it introduces **Parametric ReLU (PReLU)**, an activation function that learns the negative slope during training with negligible additional computational and parameter cost. Second, it derives Kaiming initialization (something we derived earlier this assignment), a variance-preserving initialization specifically designed for rectifier nonlinearities such as ReLU and PReLU. The theoretical analysis explains how rectification affects forward activations and backward gradients, while experiments demonstrate that the proposed initialization enables successful training of substantially deeper plain networks. The resulting PReLU network achieved a 4.94% top-5 error on ImageNet, which the authors reported as surpassing the then-reported 5.1% human-level benchmark.

A major strength is the strong connection between the theoretical variance analysis and the empirical results. However, the paper focuses primarily on plain convolutional networks and therefore does not address later architectural solutions to deep-network optimization. In particular, Batch Norm subsequently reduced the sensitivity of deep networks to initialization, making precise variance-preserving initialization less critical in some architectures, while Residual Networks (ResNets), a huge innovation in the field, made substantially deeper networks easier to optimize through skip connections. Thus, all in all, some of the practical motivation for carefully preserving activation variance became less restrictive in later architectures.

Personal Takeaway: My main takeaway is that difficulty in training deep networks is not necessarily a limitation of their representational capacity, but can instead result from optimization and signal-propagation problems. I found it particularly noteworthy that a relatively simple, theoretically motivated change to initialization enabled a 30-layer plain network to train successfully. In retrospect, later developments such as the ones mentioned previously

addressed related optimization challenges through normalization and architectural changes. This paper therefore illustrates how understanding the mathematical behavior of activations and gradients can lead directly to practical improvements in neural-network design.